

Bài 10 - Mã độc phi tập tin (Fileless Malware)

Dr. Phan The Duy - Email: duypt@uit.edu.vn

***University of Information Technology, Vietnam National University
Ho Chi Minh City***

NT230: Cơ chế hoạt động mã độc
Saturday 6th June, 2026



Chủ đề: "Fileless Malware: Tổng quan Cấu trúc và Các Khái niệm Cốt lõi cho Phân tích An ninh Mạng Nâng cao"

Làm thế nào mã độc có thể thực thi và duy trì sự hiện diện bền bỉ bằng cách tận dụng các công cụ hệ thống tin cậy mà không cần tệp tin thực thi trên đĩa?

*Meta-question: "Trong một môi trường hệ điều hành được giám sát **chặt chẽ** bởi AV/EDR, làm thế nào mã độc có thể lợi dụng các **công cụ hệ thống** (LoLBins), **dịch vụ Windows** (WMI/Registry) và **không gian bộ nhớ RAM** để thực thi mã mà vẫn **duy trì sự tàng hình** — và ranh giới giữa một **lệnh quản trị hợp lệ** với hành vi của mã độc nằm ở đâu?"*

Contents

- 1 Tổng quan về mã độc phi tập tin
- 2 Kỹ thuật trốn tránh
- 3 Cơ chế phát hiện
- 4 Các xu hướng hiện đại
- 5 Tổng kết

Phần 1: Mô hình môi đe dọa và Định nghĩa chiến lược (Mã độc không dùng tệp)

Trước khi phân tích kỹ thuật, cần hiểu sự chuyển dịch từ mã độc dựa trên tệp (Host-dependent) sang chiến thuật thực thi trực tiếp trong RAM và lưu trữ bền bỉ qua các dịch vụ hệ thống.

*Câu hỏi chính: Làm thế nào mã độc có thể đạt được mục tiêu **Zero Footprint** ($\sum TM_i \rightarrow 0$), lợi dụng **Vòng đời tấn công** (Attack Lifecycle) 5 giai đoạn để ẩn mình trong **bộ nhớ RAM** và **Registry** mà không cần tệp thực thi vật lý — và đâu là ranh giới của sự tàng hình?*

PHẦN 1: GIỚI THIỆU TỔNG QUAN VỀ MÃ ĐỘC PHI TẬP TIN

Mục Tiêu Học Tập Của Phần 1

- Hiểu rõ định nghĩa, phân loại và lịch sử tiến hóa của fileless malware theo chuẩn quốc tế (NIST/MITRE).
- Phân tích vòng đời tấn công (attack lifecycle) và tính nguy hiểm so với malware truyền thống.
- Đánh giá nghịch lý "fileless" và thách thức nghiên cứu (dựa trên memory forensics và evasion techniques).
- Áp dụng kiến thức vào phân tích thực tế (e.g., case Kovter từ nghiên cứu 2023).

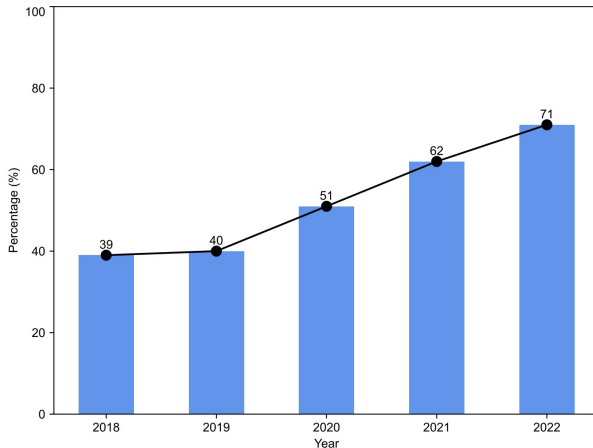
Sự chuyển dịch từ File-based sang Fileless

Traditional Malware (Host-dependent): Phụ thuộc vào tệp thực thi (PE/ELF) lưu trữ trên đĩa để kích hoạt.

Fileless Attack: Không lưu tệp thực thi độc hại lên đĩa, giảm thiểu tối đa dấu vết hoạt động (disk activity traces).

Động lực: AV truyền thống quét chữ ký (signature-based) dựa trên tệp tin; mã độc không tệp tin được thiết kế để "vô hình" trước các cơ chế này.

Sự gia tăng của mã độc phi tập tin



Hình: Thống kê sự gia tăng của tấn công mạng bằng mã độc phi mã (CrowdStrike, 2023).

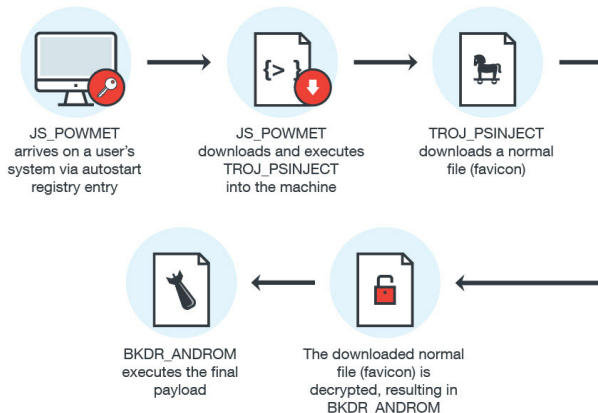
Định nghĩa học thuật về Fileless Malware

Fileless malware là loại tấn công tận dụng các tệp, ứng dụng và dịch vụ hệ thống hợp lệ để thực thi hành vi độc hại.

Cơ chế thực thi: Payload không nạp từ đĩa mà được nạp trực tiếp vào bộ nhớ thông qua các kỹ thuật script hoặc tiêm mã (injection).

Được gọi là "Bodiless malware" (Kaspersky) hoặc "Non-malware attacks" (Carbon Black).

Ví dụ quy trình lây nhiễm của mã độc phi tập tin



Hình: Quy trình lây nhiễm của mã độc phi tập tin JS_POWMET (phát hiện bởi Trend Micro).

Mục tiêu chiến lược: Tiệm cận "Zero Footprint"

Gọi TM_i là dấu vết ghi đĩa của phương thức tấn công i .

Mục tiêu của kẻ tấn công: $\sum_{i=1}^n TM_i \rightarrow 0$.

Hệ quả: Vô hiệu hóa các phương pháp quét tĩnh (Static Scanning) vốn dựa vào việc phân tích byte thô của tệp PE trên đĩa.

Thách thức pháp y: Không để lại bằng chứng vật lý (file) để phân tích sau sự cố.

Tại sao Fileless Malware nguy hiểm?

Tỷ lệ thành công: Cao gấp 10 lần so với tấn công truyền thống.

Phạm vi: Năm 2022, 71% các cuộc tấn công được phát hiện có sử dụng kỹ thuật không dùng tệp.

Vượt rào phòng thủ: 80% sản phẩm AV hàng đầu không thể phát hiện các biến thể fileless tinh vi trong các thử nghiệm thực tế.

Lợi dụng lòng tin: Núp bóng các tiến trình hệ thống như powershell.exe hoặc wmic.exe.

Phân loại theo cấu trúc hệ thống (System-hierarchy Leveraging)

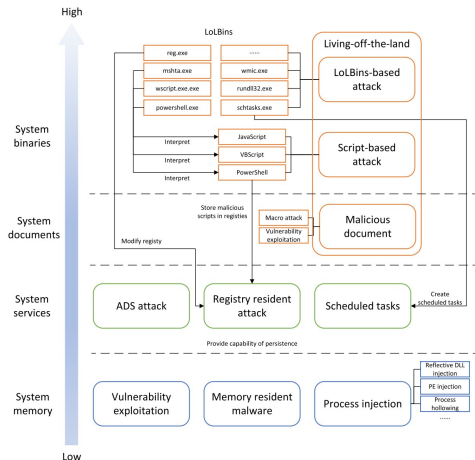
Dựa trên tầng hệ thống bị lợi dụng, Fileless malware chia thành:

Memory-based: Thực thi hoàn toàn trong RAM, không ghi đĩa (v.d: Code Red).

Service-based: Lợi dụng dịch vụ như Windows Registry, Windows Management Instrumentation (WMI), Scheduled Tasks để lưu trữ và kích hoạt mã.

LotL-based (Living-off-the-Land): Sử dụng các tệp nhị phân hợp lệ (LoLBins) hoặc tài liệu (Office/PDF) chứa script độc hại.

Phân loại theo cấu trúc hệ thống



Hình: Phân loại mã độc phi tập tin theo cấu trúc hệ thống

Threat Model Của Fileless Malware

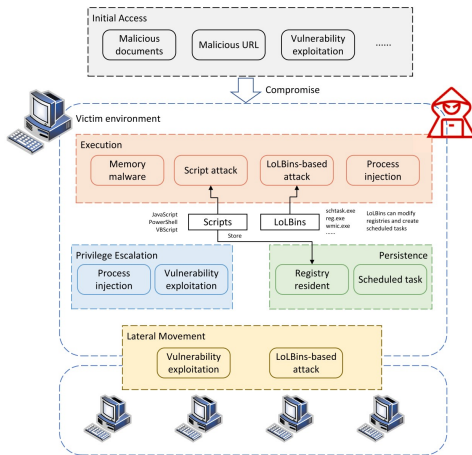
Mô Hình Đe Dọa Toàn Diện: Dựa trên nghiên cứu 2024 (Liu et al.), bao gồm các yếu tố: Actor (kẻ tấn công), Vector (delivery như phishing), Technique (injection/LotL), và Impact (persistence, evasion).

Các Thành Phần Chính:

- **Stealthiness:** Giảm dấu vết đĩa để evade AV (signature-based).
- **Difficulty in Detection:** Lợi dụng provenance graph để che giấu hành vi.
- **Research Challenges:** Tích hợp AI trong evasion (từ CrowdStrike 2023).

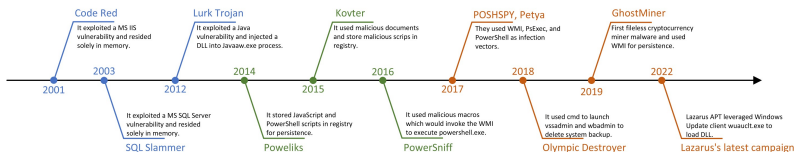
Ứng Dụng: Giúp xây dựng chiến lược phòng thủ (e.g., memory forensics như Volatility).

Threat Model Của Fileless Malware



Hình: Mô hình mối đe dọa của mã độc phi tập tin

Lịch sử tiến hóa của mã độc phi tập tin



Hình: Lịch sử tiến hóa của mã độc phi tập tin từ 2001 tới 2022.

Lịch sử tiến hóa (Giai đoạn 1: Trú ngụ bộ nhớ)

Code Red (2001): Khai thác lỗi Buffer Overflow trên MS IIS, chỉ tồn tại trong RAM dưới dạng luồng dữ liệu TCP/IP.

SQL Slammer (2003): Gói tin UDP kích thước 376 bytes tự lan truyền trực tiếp vào bộ nhớ của SQL Server.

Hạn chế: Mất đi hoàn toàn sau khi hệ thống khởi động lại (Reboot).

Lịch sử tiến hóa (Giai đoạn 2: Registry Persistence)

Poweliks (2014): Mở ra kỷ nguyên lưu trữ bền bỉ mà không cần tệp tin thực thi.

Cơ chế: Giấu JavaScript mã hóa trong Registry keys. Khi khởi động, nó gọi rundll32.exe để giải mã và nạp mã độc vào RAM.

Thách thức: Công cụ phân tích thông thường không đọc được các chuỗi non-ASCII trong registry của mã độc này.

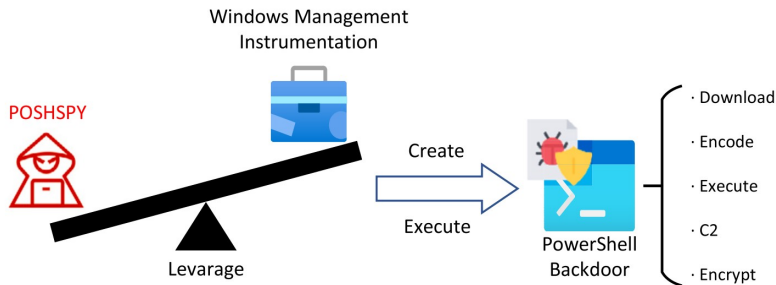
Lịch sử tiến hóa (Giai đoạn 3: LotL hiện đại)

POSHSPY (2017): Sử dụng WMI để lưu trữ backdoor và PowerShell để thực thi payload.

Tận dụng tối đa các công cụ quản trị có sẵn để che giấu hành vi trong các tác vụ hợp lệ.

Xu hướng: Trở thành vũ khí chủ đạo trong các chiến dịch APT (Advanced Persistent Threat).

Lịch sử tiến hóa của mã độc phi tập tin: POSHSPY



Hình: POSHSPY tận dụng Windows Management Instrumentation (WMI) và PowerShell để thực hiện hành vi độc hại.

Mô hình vòng đời tấn công (Attack Lifecycle)

Quy trình 5 giai đoạn được chuẩn hóa:

Delivery: Xâm nhập thông qua Exploit kits, Phishing hoặc web độc hại.

Code Injection: Tiêm mã trực tiếp vào RAM tiến trình hợp lệ.

Persistence: Thiết lập cơ chế tự khởi động qua Registry/WMI.

Execution: Chạy mã thông qua các trình thông dịch (PowerShell, JS).

Payload: Thực thi mục tiêu cuối (đánh cắp dữ liệu, mã hóa).

Giai đoạn Xâm nhập (Delivery & Infection)

Vulnerability Exploitation: Khai thác các lỗi RCE (Remote Code Execution) như EternalBlue (WannaCry) để đẩy payload trực tiếp vào RAM.

Phishing Emails: Dẫn dụ người dùng nhấn vào link chứa mã độc thực thi trực tiếp trong RAM sau khi tải.

Malicious Websites: Tự động tải mã độc vào trình duyệt thông qua các lỗ hổng ứng dụng web.

Giai đoạn Tiêm mã (Code Injection)

Cơ chế thực thi mã trong không gian địa chỉ của tiến trình khác:

PE Injection: Sao chép mã độc vào RAM của tiến trình đích và tạo luồng thực thi mới.

Reflective DLL Injection: Tự nạp DLL từ bộ nhớ mà không cần Windows Loader, tránh được việc AV giám sát các API nạp tệp truyền thống.

Process Hollowing: Thay thế phần thực thi của tiến trình hợp lệ (v.d: regsvr32.exe) bằng mã độc trong khi giữ nguyên vỏ bọc hệ thống.

Giai đoạn Duy trì (Persistence)

Làm thế nào để tồn tại sau khi máy tắt mà không có file?

Registry-Resident: Lưu mã độc dưới dạng các biến môi trường hoặc script mã hóa trong Registry Run keys.

WMI Repository: Đăng ký sự kiện hệ thống để tự kích hoạt mã khi một điều kiện (v.d: thời gian, sự kiện hệ thống) thỏa mãn.

Scheduled Tasks: Tạo các tác vụ lập lịch gọi lệnh script thay vì gọi tệp thực thi độc hại.

Giai đoạn Thực thi (Execution)

Sử dụng các công cụ "chính chủ" (LoLBins) để thực thi lệnh:

PowerShell: Quyền truy cập trực tiếp vào .NET Framework và Windows API, cực kỳ linh hoạt.

Mshta/Rundll32: Proxy thực thi các script JavaScript/VBScript mà không bị AV nghi ngờ.

Windows Management Instrumentation Command-line (WMIC): Lợi dụng để thực hiện các hành vi di chuyển ngang (Lateral Movement) và trình thám hệ thống.

Nghịch lý "Fileless": Có thực sự là không có tệp?

Câu trả lời: Không hoàn toàn.

Bản chất "không tệp" nằm ở việc không có tệp thực thi độc hại (PE file) riêng biệt.

Dấu vết vẫn tồn tại ở dạng khác:

- Các tệp tạm (temporary files) được tạo ra trong quá trình lây nhiễm.
- Bản ghi trong cơ sở dữ liệu Registry (Registry thực chất là một tệp trên đĩa).
- Các tệp tài liệu mồi (Office/PDF) chứa macro.

Kết luận: "Fileless" là một chiến thuật tấn công tàng hình, không phải là sự biến mất hoàn toàn của dữ liệu trên đĩa.

Tóm Tắt Phần 1 Và Tài Liệu Tham Khảo

Key Takeaways: Fileless malware là xu hướng evasion hiện đại, tập trung vào memory/registry/LotL, với tỷ lệ thành công cao và thách thức detection.

Tài Liệu Tham Khảo Chính:

- Kara (2023): "Fileless malware threats: Recent advances, analysis through memory forensics" (Elsevier).
- Liu et al. (2024): "A survey on the evolution of fileless attacks and detection techniques" (Computers & Security).
- Sudhakar & Kumar (2020): "An emerging threat Fileless malware: a survey and research challenges" (Springer).
- Sách: "Evasive Malware" (Cucci, 2024); "Antivirus Bypass Techniques" (Yehoshua & Kosayev, 2021); "MD MZ" (Zhussupov, 2022).

Gợi Ý Đọc Thêm: Tập trung vào case Kovter và POSHSPY cho bài tập phân tích.

Phần 2: Kỹ thuật Trốn tránh (Evasion) và Duy trì sự hiện diện (Persistence) tinh vi

Phân tích cơ chế thao túng bộ nhớ, lạm dụng dịch vụ hệ thống và chiến thuật Living-off-the-Land.

Mục tiêu: Hiểu cách mã độc đạt được trạng thái "vô hình" trước AV/EDR và duy trì quyền kiểm soát bền bỉ.

PHẦN 2: CÁC KỸ THUẬT TRỐN TRÁNH

Mục Tiêu Học Tập Của Phần 2

- Phân loại và giải thích các kỹ thuật evasion chính theo system hierarchy
- Phân tích cách thức thực hiện, ưu điểm, nhược điểm và thách thức phát hiện
- Áp dụng qua các case study thực tế và thảo luận ban đầu về phương pháp đối phó (countermeasures)
- Liên kết với chuẩn nghiên cứu quốc tế (MITRE ATT&CK, NIST Framework, CrowdStrike reports)

Phân Loại Kỹ Thuật Trốn Tránh (System Hierarchy)

Dựa trên bài báo 2024 (Liu et al.):

Memory-based: Thực thi hoàn toàn trong RAM, tránh ghi đĩa

Service-based: Lợi dụng dịch vụ hệ thống (WMI, Registry, Scheduled Tasks)

Living-off-the-Land (LotL): Sử dụng công cụ native (PowerShell, cmd.exe, LoLBins)

Other: Obfuscation, Anti-analysis, Vulnerability exploitation

Chiến thuật Living-off-the-Land (LotL)

Khái niệm: Sử dụng các tệp nhị phân, scripts và thư viện có sẵn, hợp lệ trong hệ điều hành để thực hiện hành vi độc hại

Đặc điểm cốt lõi:

- Không nạp tệp thực thi lạ lên đĩa

- Lợi dụng các công cụ đã được hệ thống “tin tưởng” (whitelisted)

Mục tiêu: Trộn lẫn hoạt động độc hại vào luồng quản trị hệ thống bình thường, gây khó khăn cho việc phân biệt

Phân loại thực thể bị lợi dụng (LoL Entities)

Dựa trên nguồn gốc và chức năng:

Pre-installed Binaries: Các tệp hệ thống mặc định trong System32 (ví dụ: mshta.exe, certutil.exe)

Post-installed Binaries: Ứng dụng hợp lệ có chữ ký số tin cậy (Microsoft Office, PsExec, v.v.)

Scripts & Interpreters: PowerShell, JavaScript, VBScript

Chi tiết LoLBins: Certutil & Bitsadmin

Certutil.exe:

Công năng gốc: Quản lý chứng chỉ số

Lạm dụng: Tải xuống payload từ URL và giải mã Base64 trực tiếp

Bitsadmin.exe:

Công năng gốc: Quản lý Background Intelligent Transfer Service

Lạm dụng: Tạo job tải payload ẩn, tự tiếp tục khi bị gián đoạn

Chi tiết LoLBins: Mshta & Rundll32

Mshta.exe: Thực thi file .hta, dùng làm proxy chạy JavaScript/VBScript mà không cần trình duyệt

Rundll32.exe: Nạp và thực thi hàm trong DLL (thường dùng để chạy mã độc trong DLL được đổi tên)

Hệ quả: Hành vi độc hại được gán cho tiến trình hệ thống hợp lệ

Ưu Nhược Điểm Của LotL

Ưu điểm:

- Stealth cao, tránh signature-based detection

- Dễ triển khai, giảm footprint trên đĩa

Nhược điểm:

- Phụ thuộc vào hệ thống cụ thể

- Dễ bị phát hiện qua logging (Sysmon, EDR)

Thách thức: Phân biệt hành vi hợp lệ và độc hại ngày càng khó

Process Injection: Thao túng không gian nhớ

Mục tiêu: Thực thi mã tùy ý trong không gian địa chỉ của tiến trình hợp lệ đang chạy

Lợi ích evasion:

Vượt tường lửa và AV bằng cách mượn danh tiến trình tin cậy

Kế thừa quyền hạn (privileges) của tiến trình đích

Dấu vết: Chỉ tồn tại trong RAM, mất khi tiến trình tắt hoặc reboot

Cơ chế PE Injection

Mở tiến trình đích bằng `OpenProcess()`

Cấp phát vùng nhớ bằng `VirtualAllocEx()`

Ghi mã độc vào vùng nhớ bằng `WriteProcessMemory()`

Tạo luồng từ xa bằng `CreateRemoteThread()`

Lưu ý: Không nạp file lên đĩa → AV dựa trên file scanning bị vô hiệu.

Reflective DLL Injection – Kỹ thuật nâng cao

Điểm khác biệt: Không dùng Windows Loader hay LoadLibrary (dễ bị giám sát)

Cơ chế: Tự triển khai hàm ReflectiveLoader trong bộ nhớ

Ưu điểm: DLL có thể truyền qua socket và nạp trực tiếp vào RAM, không bao giờ chạm đĩa

Process Hollowing (Tiến trình ma - Rỗng hóa tiến trình)

Tạo tiến trình hợp lệ (ví dụ: `svchost.exe`) ở trạng thái **Suspended**

Giải phóng vùng nhớ gốc bằng `ZwUnmapViewOfSection()`

Ghi mã độc vào vùng nhớ đã rỗng

Điều chỉnh context luồng và `ResumeThread()`

Kết quả: Mã độc chạy dưới lớp vỏ hoàn hảo của tiến trình hệ thống.

Memory-based Injection: Các kỹ thuật nâng cao không để lại dấu vết đĩa

Mục tiêu chung: Thực thi mã độc hại hoàn toàn trong RAM, tránh mọi hoạt động ghi đĩa (write disk) để bypass forensics và AV.

Kỹ thuật bổ sung:

Process Doppelganging: "Nhân bản" process mà không tạo file mới.

Atom Bombing: Sử dụng Atom Tables để inject code gián tiếp.

Threadless Injection: Inject mà không tạo thread mới.

Heap Spraying: Phân bổ memory để hỗ trợ exploit.

Shellcode Execution: Thực thi shellcode trực tiếp từ memory buffer.

Ưu điểm nhóm: Volatile (mất sau reboot), nhưng thách thức detection qua memory forensics.

Nguồn: Cucci (2024) - Evasive Malware; Yehoshua (2021) - Antivirus Bypass Techniques.

Process Doppelganging: "Nhân bản" process không ghi đĩa

Cơ chế:

Sử dụng NTFS Transactions để tạo file tạm (nhưng không commit write).

Load PE header từ memory, sau đó "transact" để tạo process mới.

Thay thế nội dung process bằng malicious code trước khi commit.

Ưu điểm: Không để lại file traces, bypass AV monitor process creation.

Nhược điểm: Phụ thuộc NTFS (Windows-specific), dễ detect qua transaction logs nếu analyst biết.

Case study: Variant của FinFisher spyware (2018) sử dụng để evade sandbox.

Tham khảo: Yehoshua & Kosayev (2021) - Antivirus Bypass Techniques, Chapter on Advanced Injection.

Atom Bombing: Inject qua Windows Atom Tables

Cơ chế:

Sử dụng Global Atom Table (kernel structure) để lưu trữ shellcode.

Gọi API như GlobalAddAtom để write code vào table.

Sử dụng APC (Asynchronous Procedure Call) để execute code từ atom table vào process đích.

Ưu điểm: Không dùng API injection tiêu chuẩn (VirtualAllocEx, CreateRemoteThread) → bypass nhiều EDR hooks.

Nhược điểm: Giới hạn kích thước shellcode (khoảng 1KB), dễ detect qua unusual atom table usage.

Case study: Talos (Cisco) report (2016) – Sử dụng trong APT attacks để inject global data.

Tham khảo: Cucci (2024) - Evasive Malware, Section on Kernel Evasion.

Threadless Injection: Inject không tạo thread mới

Cơ chế:

Sử dụng callback functions hoặc hooking (e.g., SetWindowsHookEx) để execute code.

Write shellcode vào memory đích, sau đó trigger qua event hoặc callback mà không CreateRemoteThread.

Ưu điểm: Tránh detection từ API thread creation, rất stealthy trong memory.

Nhược điểm: Phụ thuộc callback availability, khó implement ổn định.

Case study: Black Lotus Labs (2023) – Sử dụng trong fileless campaigns nhắm vào enterprise networks.

Tham khảo: Liu et al. (2024) - A survey on fileless attacks, Section on Advanced Memory Techniques.

Heap Spraying: Phân bổ memory hỗ trợ exploit

Cơ chế:

Phân bổ (spray) nhiều vùng heap với shellcode + NOP sled.

Tăng xác suất jump đến shellcode khi exploit buffer overflow/Use-After-Free.

Thực thi hoàn toàn in-memory, thường kết hợp browser exploits.

Ưu điểm: Không cần injection trực tiếp, rất hiệu quả cho RCE fileless.

Nhược điểm: Tăng memory usage → dễ detect qua anomaly monitoring.

Case study: EternalBlue variants (post-2017) – Kết hợp spraying để stable exploit.

Tham khảo: Sudhakar & Kumar (2020) - An emerging threat Fileless malware, Section on Exploitation.

Shellcode Execution: Thực thi trực tiếp từ memory buffer

Cơ chế:

Download shellcode vào memory buffer (không file).

Allocate executable memory (VirtualAlloc with PAGE_EXECUTE_READWRITE).

Jump/execute shellcode mà không process mới.

Ưu điểm: Đơn giản, không dấu vết đĩa, dễ obfuscate shellcode.

Nhược điểm: Dễ detect qua malfind (Volatility) nếu không encrypt.

Case study: Astaroth (2019) – Shellcode từ WMI query, execute in-memory.

Tham khảo: Yehoshua & Kosayev (2021) - Antivirus Bypass Techniques, Chapter on Shellcode Techniques.

Ưu Nhược Điểm Memory-Based Injection

Ưu điểm:

- Volatile – không để lại dấu vết lâu dài trên đĩa
- Kế thừa quyền hạn từ process đích

Nhược điểm:

- Mất persistence sau reboot
- Dễ bị phát hiện qua memory forensics (Volatility, Rekall)

Thách thức: Tăng sử dụng CPU/RAM có thể gây nghi ngờ

Persistence qua Windows Registry

Registry là cơ sở dữ liệu cấu hình hệ thống, tổ chức phân cấp như file system

Chiến thuật: Lưu mã độc dưới dạng chuỗi mã hóa thay vì file thực thi

Điểm kích hoạt: Sử dụng Run / RunOnce keys để gọi LoLBins khởi động mã độc

Case Study: Cơ chế lưu trữ của Poweliks

Nhúng JavaScript mã hóa vào khóa Registry

Khóa (Default) gọi rundll32.exe để thực thi JS

JS gọi PowerShell giải mã và nạp DLL từ Registry vào RAM

Sử dụng tên khóa chứa ký tự **non-ASCII** → công cụ thông thường không đọc/xóa được

Case Study: Sự tinh vi của Kovter

Phụ thuộc nặng vào Registry để duy trì tàng hình

Ghi script vào Registry sau lần chạy đầu

Dùng `mshta.exe` đọc JavaScript từ Registry

Tiêm shellcode vào tiến trình hợp lệ (ví dụ: `regsvr32.exe`)

Thường tự xóa file thực thi gốc sau khi persistence thành công

Case Study: PowerShell-Based Fileless Attacks

PowerShell dùng để download và thực thi payload trực tiếp trong RAM

Ví dụ điển hình: Emotet (2021) – inject qua PowerShell, bypass AV

Impact: Không để lại dấu vết trên đĩa, chỉ memory forensics mới phát hiện được

WMI: "Backdoor" không dấu vết

WMI là công cụ quản trị mạnh mẽ, có quyền can thiệp sâu vào hệ thống

Ứng dụng trong mã độc: Reconnaissance, lateral movement, persistence

Ưu điểm: Tạo backdoor thuần túy mà không để lại dấu vết trên đĩa

Cơ chế Persistence qua WMI

Mã độc sử dụng 3 thành phần chính:

Event Filter: Định nghĩa điều kiện kích hoạt (boot, thời gian, tiến trình cụ thể)

Event Consumer: Định nghĩa hành động (ví dụ: chạy PowerShell độc hại)

FilterToConsumerBinding: Liên kết filter và consumer

Ví dụ: POSHSPY sử dụng WMI để lưu trữ và kích hoạt backdoor bên bí.

Case Study: WMI trong Advanced Persistent Threats (APT)

Ví dụ: SolarWinds (2020) – sử dụng WMI trong chuỗi tấn công supply chain

Kỹ thuật: Tạo event filters cho persistence và lateral movement

Bài học: Kết hợp LotL + WMI tạo ra sự tàng hình cực cao

Ưu Nhược Điểm Service-Based Techniques

Ưu điểm: Persistence bền bỉ sau reboot (WMI triggers, Registry)

Nhược điểm: Dễ lộ qua system logs (Event Viewer, Sysmon)

Thách thức: Cần công cụ giám sát chuyên sâu (Sysmon, EDR)

Scheduled Tasks: Tự động hóa lây nhiễm

Lợi dụng `schtasks.exe` để lập lịch chạy mã độc

Kỹ thuật kết hợp: Lưu tham số độc hại trong Registry → tự tái tạo task

Ví dụ: Agent Tesla tạo task XML tạm thời để duy trì sự hiện diện

Alternate Data Streams (ADS): NTFS Hiding

Đặc tính NTFS: Một file có thể chứa nhiều luồng dữ liệu (streams)

Lạm dụng: Giấu .exe hoặc script vào luồng phụ của file văn bản

Ưu điểm: Không thay đổi kích thước file chính, không hiện qua lệnh `dir`

Thực thi: Dùng WMIC gọi `host.txt:malware.exe`

Mã độc dựa trên tài liệu (Document-based)

Office Macros: VBA trong Word/Excel làm loader/downloader

Cơ chế: Macro giải mã và gọi PowerShell thực thi trong RAM

PDF Attacks: JavaScript trong Adobe Reader hoặc OLE objects

Phân loại: File ban đầu có thể tồn tại, nhưng payload thực thi là fileless

Thao túng Script – PowerShell & Obfuscation

PowerShell: Truy cập trực tiếp .NET và Windows API, rất linh hoạt

Obfuscation techniques:

- Mã hóa Base64

- Chèn khoảng trắng, đảo ngược chuỗi, trộn ASCII/Unicode

Mục tiêu: Làm mã không đọc được bởi pattern-based scanners

Obfuscation and Anti-Analysis Techniques

Code Obfuscation: Base64, XOR, string concatenation

Anti-VM/Sandbox: Kiểm tra CPUID, registry keys, mouse movement

Time-Based Evasion: Delay execution nếu phát hiện debugger

Ưu Nhược Điểm Obfuscation Techniques

Ưu điểm: Làm chậm reverse engineering, bypass pattern-based AV

Nhược điểm: Tăng độ phức tạp, có thể lộ qua entropy analysis

Thách thức: Công cụ deobfuscation dựa trên ML đang phát triển mạnh

Vulnerability Exploitation: Điểm xâm nhập RAM

Khai thác Buffer Overflow, Use-After-Free để chiếm quyền điều khiển

RCE (Remote Code Execution): Đẩy payload trực tiếp vào RAM qua mạng

Ví dụ điển hình: WannaCry – EternalBlue khai thác SMB, thực thi XOR payload trong bộ nhớ

Anti-Forensic Techniques in Fileless

Wipe / clear event logs sau khi thực thi

Masquerading: Giả mạo tên tiến trình (svchost.exe, explorer.exe)

Encrypt payload trước khi inject

Technique	Countermeasure
Wipe Logs	SIEM / centralized logging
Masquerading	Process tree analysis
Encryption	Memory pattern scanning

So sánh khả năng trốn tránh: Traditional vs Fileless

Đặc điểm	Mã độc truyền thống	Mã độc không dùng tệp
Tệp thực thi	Có (PE, DLL trên đĩa)	Không (chỉ trong RAM)
Tiền trình	Tiền trình lạ dễ nhận diện	Núp bóng tiến trình hệ thống
Dấu vết đĩa	Nhiều dấu vết vật lý	Tối thiểu ($\sum TM_i \rightarrow 0$)
Cơ chế phát hiện chính	Quét chữ ký (signature)	Phân tích hành vi / bộ nhớ
Độ phức tạp	Trung bình	Rất cao

Thách thức phát hiện từ góc nhìn kẻ tấn công

Không để lại dấu vết vật lý trên đĩa → các công cụ quét tệp truyền thống gần như vô hiệu

Hành vi gần giống hết với hoạt động quản trị hệ thống hợp lệ (PowerShell, WMI, rundll32...)

Dữ liệu tồn tại chủ yếu trong bộ nhớ → mất đi nhanh chóng sau reboot hoặc kill process

Khả năng chống phân tích môi trường (anti-VM, anti-sandbox) làm chậm quá trình reverse

Tăng độ phức tạp của mã (obfuscation) khiến việc xây dựng signature/pattern trở nên cực kỳ khó khăn

Nhiều kỹ thuật được thiết kế đặc biệt để khai thác khoảng cách giữa signature-based và behavior-based detection.

Tại sao fileless vẫn khó bị phát hiện (tóm tắt)

Không tạo tệp PE độc hại

Không gọi LoadLibrary /
CreateFile đáng ngờ

Không để lại artifact trên
đĩa (file, prefetch,
shimcache...)

Núp bóng dưới các tiến
trình / dịch vụ hệ thống
tin cậy

Lợi dụng các công cụ đã
được whitelisted

Dễ dàng obfuscate script /
shellcode

Có thể tự xóa dấu vết
(self-deleting, log wiping)

Persistence dựa vào logic
hệ thống (WMI, Registry)
thay vì file

→ Kẻ tấn công tận dụng chính niềm tin của hệ thống vào các thành phần native.

Hạn chế nội tại của các kỹ thuật evasion

Memory-based: Mất persistence sau reboot hoặc kill process

LotL: Phụ thuộc vào cấu hình hệ thống (có thể bị disable PowerShell, WMI...)

Obfuscation: Tăng kích thước mã, entropy cao, có thể gây chậm trễ thực thi

Registry/WMI persistence: Dễ bị phát hiện nếu analyst chủ động kiểm tra các key/event filter

Anti-analysis: Nếu môi trường phân tích tinh vi hơn (bare-metal, time-aware sandbox) thì dễ bị lộ

Mọi kỹ thuật đều có trade-off giữa độ tàng hình và độ bền vững.

So sánh lại – Traditional vs Fileless (Trade-off)

Tiêu chí	Mã độc truyền thống	Fileless / Memory-resident
Dấu vết trên đĩa	Rất nhiều	Tối thiểu đến không
Persistence sau reboot	Dễ (file + registry)	Khó hơn (phụ thuộc logic hệ thống)
Khả năng ẩn tiến trình	Thấp	Rất cao
Khả năng chống reverse	Trung bình	Cao (obfuscation + anti-analysis)
Độ phức tạp triển khai	Trung bình	Cao
Độ khó phát hiện (hiện tại)	Thấp – trung bình	Cao

Fileless đánh đổi tính bền vững để đạt độ tàng hình tối đa.

Tổng kết Phần 2 – Góc nhìn kẻ tấn công

- + Fileless malware đạt được sự tàng hình vượt trội bằng cách khai thác niềm tin hệ thống vào các công cụ native (LotL)
- + Persistence chuyển từ file thực thi sang các thực thể logic (Registry, WMI, Scheduled Tasks, ADS...)
- + Kỹ thuật injection và obfuscation giúp mã độc tồn tại trong bộ nhớ với mức độ ẩn danh cao
- + Mọi kỹ thuật đều có trade-off: tàng hình $\leftrightarrow$ độ bền vững $\leftrightarrow$ độ phức tạp triển khai
- + Ranh giới giữa hành vi quản trị hợp lệ và hành vi tấn công ngày càng mờ nhạt

→ Đây chính là thách thức lớn nhất đối với phòng thủ hiện đại

Teaser – Chuyển sang Phần 3

Từ góc nhìn kẻ tấn công...
...sang góc nhìn người phòng thủ

Phần 3 sẽ tập trung vào:

Cơ chế giám sát hành vi hệ thống

Phân tích bộ nhớ (memory forensics)

Phát hiện dựa trên hành vi và provenance

Các công cụ và chiến lược đối phó thực tế

Chuẩn bị sẵn sàng để chuyển từ "làm sao để ẩn" sang "làm sao để tìm ra".

Tài liệu tham khảo – Phần 2

Sách (chủ yếu về evasion & development):

Evasive Malware – Kyle Cucci

Antivirus Bypass Techniques – Nir Yehoshua & Uriel
Kosayev

MD MZ – Zhussupov Zhassulan (COCOMELONC)

The Antivirus Hacker's Handbook – Koret & Bachaalany

Bài báo khảo sát:

Liu et al. (2024) – A survey on the evolution of fileless
attacks

Kara (2023) – Fileless malware threats: Recent
advances...

Sudhakar & Kumar (2020) – An emerging threat Fileless
malware...

Tài liệu chuyên sâu về detection sẽ được giới thiệu ở Phần 3.

Q&A – Phần 2

Mời các bạn đặt câu hỏi về:

Các kỹ thuật evasion cụ thể (LotL, injection, WMI, obfuscation...)

Case study (Poweliks, Kovter, POSHSPY, Emotet...)

Trade-off giữa tàng hình và độ bền vững

Những hạn chế nội tại của các kỹ thuật fileless hiện nay

Phần 3: Chiến lược Giám sát, Phân tích, Phát hiện và Giảm thiểu Mã độc không dùng tệp

Từ phân tích pháp y bộ nhớ đến đồ thị nguồn gốc để nhận diện "bóng ma" trong hệ thống.

Câu hỏi cốt lõi: Làm thế nào để phát hiện hành vi tấn công khi không có tệp thực thi nào trên đĩa?

PHẦN 3: CHIẾN LƯỢC GIÁM SÁT, PHÂN TÍCH, PHÁT HIỆN VÀ GIẢM THiểu FILELESS MALWARE

Mục tiêu học tập của Phần 3

Hiểu rõ các phương pháp giám sát và phân tích fileless theo khung NIST / MITRE ATT&CK (TA0005 Defense Evasion, TA0003 Persistence).

So sánh ưu nhược điểm của rule-based, memory forensics và behavioral detection.

Áp dụng thực tế các công cụ: Volatility, Sysmon, YARA, Sigma.

Đề xuất chiến lược mitigation doanh nghiệp và nhận diện thách thức hiện tại.

45 phút: Lý thuyết 60%, minh họa + case study 30%, Q&A + bài tập ngắn 10%.

Tổng quan các phương pháp phát hiện

Phương pháp truyền thống dựa trên chữ ký tệp (static file scanning) gần như thất bại trước fileless. Các chiến lược hiện đại chia thành 3 nhóm chính:

Rule-based detection: YARA (memory patterns), Sigma (log events)

Memory forensics: Phân tích RAM dump để tìm artifact in-memory

Behavioral & graph-based detection: Theo dõi hành vi, chuỗi sự kiện, provenance graph

Chú ý: ML/DL và AI nâng cao sẽ được trình bày chi tiết ở Phần 4 (xu hướng hiện đại).

So sánh 3 nhóm phương pháp phát hiện

Phương pháp	Ưu điểm
Rule-based (YARA/Sigma)	Nhanh, chính xác với known IOC
Memory Forensics	Phát hiện artifact ẩn trong RAM
Behavioral / Provenance Graph	Phát hiện unknown attacks, ngữ cảnh rộng

Nguồn: Dựa trên Liu et al. (2024), Kara (2023), Sudhakar (2020).

Rule-based: YARA trong phân tích bộ nhớ

Vai trò: Pattern matching linh hoạt nhất cho memory dump và fileless artifacts

Ứng dụng: Tìm chuỗi, regex, byte patterns đặc trưng của shellcode, PowerShell injection

Ưu điểm: Độ chính xác cao với known variants, tích hợp dễ vào pipeline forensics

Hạn chế: Không realtime, dễ bị bypass bởi polymorphism/obfuscation nặng

Công cụ: YARA + Volatility plugin.

Ví dụ YARA Rule phát hiện PowerShell injection

Rule cơ bản phát hiện pattern phổ biến trong memory:

```
rule Fileless_PowerShell_Injection {
  meta:
    description = "Detect common PowerShell
                  injection patterns"
    author      = "Adapted from Monnappa K A"
  strings:
    $ps1 = "powershell" ascii nocase
    $ps2 = "Invoke-Expression" ascii nocase
           wide
    $ps3 = "IEX" ascii nocase
    $shell = "shellcode" ascii nocase
  condition:
    2 of ($ps*) and $shell
}
```

Ứng dụng: Chạy trên memory dump từ Volatility để match injected code.

Rule-based: Sigma và giám sát nhật ký hệ thống

Vai trò: Chuẩn hóa signature cho log events (tương tự YARA cho file)

Cơ chế: Phát hiện lạm dụng LoLBins qua command-line arguments

Event ID cần giám sát chặt chẽ:

4688: Process creation (cha là PowerShell/cmd)

4104: PowerShell Script Block Logging

7045: New service installation (WMI persistence)

11: File creation in suspicious paths

Công cụ: Sysmon + Sigma rules → SIEM (Splunk/ELK).

Quy trình Pháp y Bộ nhớ (Memory Forensics)

Thu thập: Dump RAM bằng DumpIt, FTK Imager, Magnet RAM Capture

Chuyển đổi: Chuẩn bị profile phù hợp (Win10x64, etc.)

Phân tích: Sử dụng Volatility / Rekall để liệt kê:

- Processes (pslist, psscan)

- DLLs injected (malfind)

- Network connections (netscan)

- Registry artifacts (hivelist, printkey)

So sánh: Đối chiếu với baseline sạch để tìm anomaly

Công cụ Memory Forensics chính

Volatility 3: Framework mã nguồn mở, hỗ trợ Windows/Linux/macOS

Rekall: Tập trung live forensics và kernel object analysis

MemProcFS: Giao diện file system ảo cho memory dump (rất mạnh)

Ví dụ lệnh phổ biến:

```
vol.py -f dump.mem windows.pslist  
vol.py -f dump.mem windows.malfind
```

Nguồn: Monnappa K A – Learning Malware Analysis.

Case Study: Phát hiện Kovter qua Memory Forensics

Hành vi: Registry hiding + PowerShell injection

Phân tích: Dump RAM → Volatility → phát hiện:

- Non-ASCII registry keys chứa script

- Injected code trong regsvr32.exe

- Suspicious PowerShell child processes

Kết quả: Xác định full attack chain mà không cần file trên đĩa

Bài học: Memory forensics giảm tải xử lý so với static unpacking

Nguồn: Kara (2023) – Real case application.

Phát hiện qua API Hooking và Behavioral Monitoring

Giám sát API quan trọng:

VirtualAllocEx, WriteProcessMemory,
CreateRemoteThread
NtCreateSection, NtMapViewOfSection (reflective
injection)

Công cụ: ProcMon, API Monitor, Microsoft Defender ATP

Logic phát hiện: Phát hiện chuỗi API bất thường (ví dụ:
rundll32 gọi WMI + network outbound)

Hạn chế: Overhead cao, khó triển khai ở quy mô lớn

Thách thức chính trong phát hiện fileless

False Positives cao: PowerShell/WMI thường dùng bởi admin hợp pháp

Volatility của memory: Dữ liệu mất sau reboot hoặc process termination

Anti-forensics: Attacker wipe logs, tamper ETW, sleep nếu phát hiện analysis env

Scale: Khó phân tích hàng triệu endpoints realtime

Obfuscation: Script PowerShell phức tạp làm giảm hiệu quả rule-based

Chiến lược giảm thiểu – Doanh nghiệp

Patch management: Vá nhanh RCE (EternalBlue, PrintNightmare...)

Application Control: AppLocker / WDAC whitelist binaries

PowerShell hardening: Constrained Language Mode, Script Block Logging

LoLBins restriction: Block/audit certutil, bitsadmin, mshta (qua GPO)

Macro/script control: Disable macro từ internet, chỉ cho phép signed scripts

User awareness: Đào tạo chống phishing – vector khởi đầu chính

Tổng kết Phần 3

Fileless malware đòi hỏi chuyển từ file-based sang **behavioral + memory-based** detection

Memory forensics (Volatility) và log monitoring (Sysmon + Sigma) là nền tảng cốt lõi

Provenance graph và API monitoring giúp phát hiện chuỗi tấn công phức tạp

Mitigation tập trung vào hardening, patch và user training

Thách thức lớn nhất: Cân bằng giữa false positive và độ nhạy

Teaser Phần 4: Xu hướng mới – AI trong cả evasion và detection.

Q&A và gợi ý bài tập

Thảo luận: Trong môi trường doanh nghiệp Việt Nam, bạn ưu tiên triển khai phương pháp nào trước? Tại sao?

Bài tập gợi ý (tự làm hoặc nhóm):

Dùng Volatility phân tích memory dump mẫu (có sẵn trên GitHub Volatility samples)

Viết 1 Sigma rule detect WMI persistence

Report ngắn: So sánh ưu nhược điểm memory forensics vs behavioral monitoring

Tài liệu tham khảo chính – Phần 3

Liu et al. (2024) – A survey on fileless attacks and detection techniques

Kara (2023) – Fileless malware threats: memory forensics approach

Sudhakar & Kumar (2020) – An emerging threat: Fileless malware survey

Monnappa K A – Learning Malware Analysis (Volatility, tools)

Mohanta & Saldanha – Malware Analysis and Detection Engineering

Cucci – Evasive Malware (behavioral detection)

Chuẩn tham chiếu: MITRE ATT&CK – Defense Evasion (TA0005), Persistence (TA0003).

Phần 4: Các Xu Hướng Hiện Đại của Fileless Malware

(2024–2026): AI, Agentic AI, Cloud-Native và Chuyển Dịch Trong Cybersecurity

Từ memory-resident → AI-driven autonomous attacks;

Định hướng nghiên cứu tiếp theo

PHẦN 4: CÁC XU HƯỚNG HIỆN ĐẠI CỦA FILELESS MALWARE

Mục tiêu học tập của Phần 4

Hiểu rõ các xu hướng fileless malware mới nhất (2024–2026), đặc biệt tích hợp AI/Agentic AI

Phân tích cuộc đua hacker AI vs. defender AI qua case study thực tế

Đánh giá tiềm năng tấn công, rủi ro, nhu cầu threat hunting và tự động hóa

Xác định khoảng trống nghiên cứu và định hướng nghiên cứu tiếp theo

Tập trung vào chuyển dịch: file-based → memory → cloud → AI/agentic

Tổng quan xu hướng fileless 2024–2026

Tăng trưởng mạnh: Fileless chiếm $>70\%$ APT attacks (CrowdStrike 2025 report)

Chuyển dịch chính: Memory-resident $\rightarrow$ Cloud-native $\rightarrow$ AI-augmented $\rightarrow$ Agentic autonomous

Tích hợp AI: LLM-generated evasion, polymorphic code, self-adapting agents

Mở rộng nền tảng: Windows $\rightarrow$ Linux (eBPF), macOS, container/Kubernetes

Living-off-the-Cloud (LoTC) và hybrid AI-human attacks

AI-powered Fileless Malware

Sử dụng LLM (GPT-4o, Claude 3.5, Llama-3) để tự động tạo:

- Polymorphic shellcode và obfuscated scripts

- Custom evasion techniques bypass EDR

- Adaptive persistence mechanisms

Tiềm năng: Tạo hàng nghìn biến thể zero-day trong giây

Rủi ro: Giảm barrier entry cho hacker novice

Agentic AI: Malware tự hành động (Autonomous Agents)

Đặc điểm chính: Malware như "agent" với reasoning loop (tương tự AutoGPT, LangChain)

Khả năng:

- Tự reconnaissance và decision-making

- Tự chọn evasion tactic dựa trên môi trường

- Tự lan rộng và exfiltrate data

- Tự xóa dấu vết nếu bị nghi ngờ

Tiềm năng: Không cần C2 tĩnh, hoạt động offline

Cuộc đua Hacker AI vs. Defender AI

Hacker AI: Tạo evasion polymorphic, agentic attacks tự thích nghi

Defender AI: ML-based anomaly detection, AI threat hunting agents

Chuyển dịch: Từ passive signature → active AI confrontation

Rủi ro: Adversarial AI – hacker "poison" defender models

Nhu cầu: Chủ động threat hunting + SOAR (Security Orchestration, Automation, Response)

Case Study 1: SolarWinds (2020, updated 2024)

Fileless component: DLL injection + WMI persistence

AI integration (2024 variant): LLM-generated polymorphic code để bypass EDR

Hacker AI: Tự động recon supply chain

Defender AI: ML detect anomalies in build pipeline
(CrowdStrike Falcon)

Bài học: Nhu cầu threat hunting proactive trong supply chain

Case Study 2: MOVEit Transfer (2023)

Fileless exploit: SQL injection → in-memory execution

Hacker AI: Auto-generated exploit scripts via GPT

Defender AI: Behavioral analytics detect lateral movement

Rủi ro: Ransomware-as-a-Service (RaaS) kết hợp AI

Nhu cầu: Tự động hóa response (SOAR) để isolate nhanh

Case Study 3: Volt Typhoon (APT China, 2024)

Fileless tactics: LotL + eBPF rootkit trên Linux

Hacker AI: Agentic recon tự động (stealthy C2)

Defender AI: Graph-based hunting detect anomalies

Cuộc đua: Hacker bypass ML models; defender dùng federated learning

Bài học: Chủ động săn tìm threat từ không gian mạng (OSINT + AI)

Case Study 4: FakeUpdate (2024)

Fileless vector: Browser hijacking → memory injection

Hacker AI: LLM tạo fake update scripts obfuscated

Defender AI: NLP analyze script semantics realtime

Rủi ro: Tấn công hàng loạt qua phishing AI-generated

Nhu cầu: Tích hợp SOAR để automate isolation/quarantine

Case Study 5: Agentic Ransomware (2025–2026)

Giả định: LockBit variant với agentic AI (self-propagating)

Hacker AI: Tự quyết định encrypt targets, negotiate ransom

Defender AI: Autonomous hunting agents counter-attack

Cuộc đua: AI vs. AI – hacker poison data; defender use deception tech

Bài học: Nhu cầu hybrid hunting (human + AI + automation)

Fileless trên Linux: eBPF-based Attacks

eBPF lạm dụng: Hide processes, network hooks kernel-level

Tiềm năng: Rất mạnh trên cloud/server Linux

Rủi ro: Khó detect bằng EDR truyền thống

Chuyển dịch: Từ Windows-centric → multi-platform

Cloud-native Fileless Attacks

Serverless exploitation: Malicious Lambda functions

Container evasion: Runtime hooking, malicious sidecars

Tiềm năng: Scale lớn, tự động deploy

Rủi ro: Lateral movement nhanh trong Kubernetes

Living-off-the-Cloud (LoTC)

Lạm dụng cloud CLI: aws, az, gcloud cho persistence

Ví dụ: aws ec2 run-instances → spawn malicious instances

Tiềm năng: Evade on-prem EDR

Rủi ro: Exfiltrate data qua cloud storage

Supply-chain Attacks kết hợp Fileless

Tấn công qua npm/PyPI: Malicious packages → in-memory exec

AI role: Auto-generate poisoned dependencies

Tiềm năng: Infect hàng triệu users

Rủi ro: Khó trace trong CI/CD pipelines

Rủ ro lớn nhất cho defender

AI accelerate malware dev: 100x faster variants

Agentic indistinguishability: Benign vs. malicious agents

Evasion sophistication: Bypass ML detectors

Traceability loss: No static C2, self-destructing

Expanded attack surface: Cloud, IoT, multi-platform

Thách thức phát hiện Agentic AI malware

Dynamic adaptation: Behavior changes per env

Reasoning evasion: Predict avoid detection

Offline operation: No network indicators

Adversarial robustness: Poison defender AI

Nhu cầu Threat Hunting & SOAR

Proactive hunting: Human-AI hybrid teams scan anomalies

SOAR: Automate response (isolate, remediate)

Beyond monitoring: OSINT from cyber space (dark web, forums)

Chuyển dịch: Từ reactive → predictive defense

Case Study: Threat Hunting in 2025 Incident

Incident: Agentic fileless in healthcare network

Hunting: AI agent scan logs + human verify

SOAR: Auto-quarantine infected nodes

Bài học: Kết hợp OSINT (X/Twitter intel) để predict attacks

Khoảng trống nghiên cứu (1)

Dataset scarcity: Thiếu public data cho Agentic AI malware

Framework evaluation: Chưa có benchmark cho AI evasion robustness

Real-time detection: Cho autonomous agents ở scale lớn

Standardization: "Agent Behavior" trong MITRE ATT&CK

Khoảng trống nghiên cứu (2)

Provenance graph in cloud: Scale & efficiency

eBPF forensics: Detection tools cho kernel-level fileless

Adversarial ML: Counter attacker AI poisoning

Privacy in hunting: Federated learning cho shared intel

Hướng nghiên cứu tương lai

Phát triển AI defender agents: Autonomous hunting
Hybrid SOAR + OSINT: Automate cyber space monitoring
Explainable AI for detection: Transparent decisions
Benchmark datasets: Simulated Agentic attacks
Global collaboration: Shared threat intel platforms

Chuyển dịch lớn trong Cybersecurity

Từ static signatures → AI-native dynamic defense

Defender shift: Proactive hunting + automated SOAR

Tăng CSPM (Cloud Security Posture Management)

Hybrid human-AI: Cuộc đua cân bằng

Tổng kết Phần 4 & Q&A

Xu hướng: AI/Agentic làm thay đổi cuộc chơi fileless

Cuộc đua: Hacker AI accelerate → Defender cần proactive hunting + SOAR

Định hướng nghiên cứu: Dataset, frameworks, adversarial robustness

Thảo luận: Bạn sẽ nghiên cứu gì tiếp theo về AI in cybersecurity?

Phần 5: Tổng kết bài học

Ôn tập các khái niệm cốt lõi, bài tập thực hành và định hướng nghiên cứu tiếp theo.

Fileless là bài học về sự chuyển dịch từ 'tìm tộp' sang 'hiểu hành vi'.

Tóm tắt: Bản chất và Chiến thuật cốt lõi của Fileless Malware

Định nghĩa chiến lược: Fileless là mã độc không lưu trữ trên đĩa, thực thi trong RAM, lợi dụng tiến trình hợp lệ để tàng hình (host-independent, bypass AV signature).

Mục tiêu tối thượng: Triệt tiêu dấu vết đĩa ($\sum TM_i \rightarrow 0$), chuyển trọng tâm tấn công từ file-based sang memory/service-based.

Vũ khí Living-off-the-Land (LotL): Lợi dụng LoLBins (PowerShell, WMI, certutil.exe, mshta.exe) để hòa lẫn hành vi độc hại vào quản trị hệ thống.

Lưu trú bền bỉ (Persistence): Sử dụng Registry (như Poweliks/Kovter), WMI Event Consumers, hoặc Scheduled Tasks để "sống sót" sau reboot mà không cần file thực thi.

Nguồn chính: Liu et al. (2024) và Kara (2023) – Nhấn mạnh sự chuyển dịch từ traditional malware.

Tóm tắt: Phát hiện, Giảm thiểu và Thách thức Nghiên cứu

Pháp y bộ nhớ (Memory Forensics): Phương pháp cốt lõi để "chụp" artifact trong RAM (Volatility, Rekall), phát hiện injection và hidden processes.

Phát hiện dựa trên Hành vi: Sử dụng Sysmon/ETW giám sát API hooking, kết hợp Sigma/YARA cho log và memory patterns.

Giảm thiểu (Mitigation): Hardening (AppLocker, PowerShell Constrained Mode), patching RCE, và đào tạo chống phishing.

Thách thức hiện hữu:

False positives cao: Khó phân biệt admin vs. malicious behavior.

Anti-forensics và obfuscation ngày càng tinh vi.

Bùng nổ dữ liệu log: Cần tối ưu hóa realtime analysis.

Định hướng: Tích hợp AI để giải quyết thách thức này.

Thông điệp cốt lõi

"Trong kỷ nguyên Fileless, chúng ta không còn đi tìm những 'tập tin lạ', mà chúng ta đi tìm những 'hành vi lạ' của những 'công cụ quen thuộc'. Sự tin cậy đôi khi chính là kẻ hở lớn nhất."

Cách đối phó mã độc phi tập tin: Hãy bắt đầu từ forensics cơ bản để xây dựng nền tảng nghiên cứu quốc tế.

Bài tập 1: Phân tích Case Study (Kovter Malware)

Mô tả: Dựa trên bài báo Kara (2023), phân tích attack chain của Kovter: Từ delivery → injection → persistence qua Registry.

Yêu cầu:

So sánh ưu nhược điểm evasion của Kovter so với Poweliks.

Đề xuất 2 phương pháp detection (memory forensics + behavioral).

Định hướng nghiên cứu: Làm thế nào cải thiện để detect non-ASCII registry keys realtime?

Hình thức: Báo cáo nhóm 5–7 trang, trình bày 10 phút.

Chuẩn quốc tế: Áp dụng MITRE ATT&CK framework (TA0005).

Bài tập 2: Phân tích So sánh Phương pháp Phát hiện

Mô tả: So sánh rule-based (YARA/Sigma) vs. memory forensics (Volatility) vs. provenance graph trong phát hiện LotL attacks.

Yêu cầu:

Đánh giá hiệu quả, false positives, và scalability trên enterprise environment.

Phân tích thách thức với AI evasion (teaser Phần 4).

Định hướng nghiên cứu: Đề xuất hybrid model kết hợp 3 phương pháp để giảm false positives.

Hình thức: Essay cá nhân 3–5 trang, thảo luận lớp.

Tham khảo: Liu et al. (2024) – Focus on research gaps.

Bài tập 3: Phân tích Đánh giá Rủi ro và Mitigation

Mô tả: Đánh giá rủi ro fileless trong cloud environment (AWS/Azure), dựa trên LoTC.

Yêu cầu:

Xác định 3 vector tấn công chính và mitigation (hardening, monitoring).

Liên hệ với xu hướng Agentic AI: Rủi ro tương lai và cách mitigate.

Định hướng nghiên cứu: Làm thế nào tích hợp OSINT vào threat hunting cho fileless?

Hình thức: Presentation nhóm + slide deck 10–15 slide.

Chuẩn: Sử dụng NIST CSF framework để đánh giá.

Bài tập 4: Coding – PoC Process Injection (Offense & Defense)

Mô tả: Xây dựng PoC đơn giản (high-level, không malicious) minh họa PE Injection.

Offense: Viết script Python (dùng ctypes) giả lập injection vào process dummy (e.g., notepad.exe).

Defense: Triển khai hook (dùng API Monitor) để detect injection, hoặc Volatility script phân tích dump.

Định hướng nghiên cứu: Mở rộng PoC để test với obfuscation – Đề xuất cách cải thiện detection cho reflective injection.

Hình thức: Code repo GitHub + demo video 5 phút. Lưu ý: Chỉ môi trường lab ảo.

Bài tập 5: Coding – Obfuscation PowerShell Script (Offense & Defense)

Mô tả: Tạo script PowerShell đơn giản (e.g., system recon) và obfuscate nó.

Offense: Sử dụng base64, string concatenation, Unicode mixing để obfuscate script.

Defense: Viết Python tool (dùng regex/NLP) để deobfuscate và detect malicious intent.

Định hướng nghiên cứu: Tích hợp ML (Word2Vec) vào deobfuscator – Test với LLM-generated obfuscation.

Hình thức: Jupyter notebook + report 5 trang. Nhấn mạnh ethical use.

Định hướng nghiên cứu tiếp theo cho Học viên Cao học ATTT

Hướng 1: Phát triển hybrid detection model: Kết hợp memory forensics + provenance graph + ML để giảm false positives.

Hướng 2: Nghiên cứu Agentic AI malware: Xây dựng benchmark dataset và test adversarial robustness.

Hướng 3: Fileless trong cloud: Thiết kế SOAR framework cho LoTC attacks, tích hợp OSINT từ cyber space.

Gợi ý: Publish tại hội nghị quốc tế (Black Hat, USENIX Security) hoặc journal (Computers & Security).

Hãy bắt đầu từ PoC thực tế để nghiên cứu và xây dựng paper.

Tài liệu tham khảo bổ sung cho Nghiên cứu Tiếp theo

- + Liu, S., Peng, G., Zeng, H., & Fu, J. (2024). A survey on the evolution of fileless attacks and detection techniques. *Computers & Security*, 137, Article 103653.
- + Kara, İ. (2023). Fileless malware threats: Recent advances, analysis approach through memory forensics and research challenges. *Expert Systems with Applications*, 213, Article 119133.
<https://doi.org/10.1016/j.eswa.2022.119133>
- + Sudhakar, & Kumar, S. (2020). An emerging threat Fileless malware: A survey and research challenges. *Cybersecurity*, 3(1), Article 1.
- + Cucci, K. (2024). *Evasive Malware: A Field Guide to Detecting, Analyzing, and Defeating Advanced Threats*. No Starch Press.
- + Yehoshua, N., & Kosayev, U. (2021). *Antivirus Bypass Techniques: Learn Practical Techniques and Tactics to Combat, Bypass, and Evade Antivirus Software*. Packt Publishing.

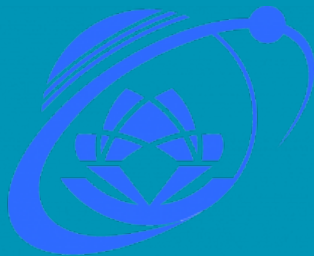
Gợi ý đọc: Chương về Evasion Techniques trong tài liệu

MD MZ (Zhussupov Zhassulan, 2022) – Tập trung phần coding PoC, các kỹ thuật obfuscation, injection và persistence thực hành.

Cảm ơn các bạn đã theo dõi!

Chúc thành công trong nghiên cứu và ứng dụng thực tế về fileless malware đúng mục đích.

Liên hệ: duypt@uit.edu.vn.



UIT

TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ THÔNG TIN